

Beevil Knievel

Technical Explainer — Ansys Simulation, PCB Design, TinyML & Cloud Model Training

Prepared for faculty review | github.com/atharveeee-netizen/beevel-knievel

1. What the System Does

- Beehive monitoring device that sits inside the hive to track colony health
- Listens to acoustic signature of bees + tracks temperature
- Runs a small AI model on-chip (TinyML) for instant local classification
- Periodically sends data via LoRa → gateway → AWS
- Cloud model (more powerful) determines: healthy / swarming / starving / varroa mites, etc.
- Beekeeper gets SMS alert if something looks wrong

Three engineering phases, built in order:

- Ansys simulation (board + enclosure)
- PCB design (based on simulation results)
- ML model training (TinyML + cloud)
- **Data flow:** Hive sensors → Edge AI → LoRa Gateway → AWS Cloud → Dashboard/SMS alert to beekeeper
- **Packet path:** Hive node → Gateway Wio-E5 (RF demodulation) → UART → Gateway ESP32 (formats JSON) → AWS (HTTPS/TLS 1.2)

2. Simulating the Board & Enclosure in Ansys

- Purpose: catch design flaws (antenna, power, thermal) before fabrication
- Three separate simulation studies

2.1 Antenna Performance

- 86 mm quarter-wave wire antenna, using STM32WLE5 built-in LoRa radio
- Tuned for 868/915 MHz
- Goal: reliable signal through wooden hive wall, 4 km range target
- Tool: HFSS (Ansys Electronics Desktop)
- Process: imported 2-layer PCB layout, modeled antenna geometry, assigned FR4/copper/wood material properties, radiation boundary, 50 ohm port

Results:

- Return loss dip below -10 dB at 868 MHz
- Omnidirectional far-field pattern
- Gain above 2 dBi
- Confirms RF energy is radiating properly, not reflecting back into chip

2.2 Power Rail Stability During Transmission

- Concern: radio TX burst could sag 3.3V rail → MCU brownout/reset
- Tool: SIwave + Nexxim circuit solver
- Modeled: TPS7A02 regulator + decoupling capacitors, board's power/ground planes
- Simulated current profile: 0.3 μ A sleep → 120 mA TX burst (60 ms), over 100 ms window

Results:

- Rail never dropped below ~3.22V
- PDN impedance stayed under 1.375 ohms across tested frequency range
- Comfortable stability margin on battery power

2.3 Thermal Impact on Temperature Sensor

- Concern: self-heating (battery/MCU) could skew temperature readings
- Tool: Icepak
- Modeled: 3D CAD of 60×50 mm enclosure + PCB, realistic heat loads, ambient air at 35°C (hive-like)

Results:

- Battery reached ~38°C locally
- Ambient sensor footprint stayed within 0.1°C of surrounding air
- Well within required accuracy

Outcome

- All three simulations gave enough confidence to move to fabrication without repeated physical prototype iterations

3. Designing & Building the PCB

- Core chip: STM32WLE5JC (Cortex-M4 MCU + sub-GHz LoRa radio in one package)
- No separate radio module needed → smaller, more power-efficient board

Repository/hardware organized into:

- Schematics (power regulation, sensor wiring, RF matching network)
- Full BOM (part numbers, pricing, pinout map)
- 3D CAD for enclosure (same 60×50 mm used in Icepak study)
- 2-layer FR4 board with dedicated 3.3V/GND planes + controlled impedance trace to antenna pad
- **Design loop:** schematic → simulate in Ansys → adjust layout → simulate again (iterative, not linear)

4. Training the Models: TinyML & Cloud

- Trained on **real** bee colony recordings (Nolasco & Benetos, 2018 Zenodo dataset) — not synthetic

Data integrity note:

- Placeholder/synthetic data + rule-based heuristic had crept in early on

- Team audited, quarantined synthetic data, retrained fully on verified recordings

Training setup:

- Fixed random seed
- Stratified split: ~68% train / 16% validation / 16% test
- Feature scaling fit only on training data (no leakage)

4.1 Audio → Features

- 2-second audio windows processed via sliding FFT
- 4 spectral bands: ventilation, swarming, distress calls, background noise
- Combined with thermal slope (from temperature sensor)
- Result: compact feature vector used by both models

4.2 TinyML Model (On-Chip)

- Small quantized 1D convolutional neural network
- Runs on STM32WLE5's Cortex-M4 core — works fully offline
- Performance: ~82% accuracy, macro F1 ~0.83, AUROC 1.0
- Size: only 803 parameters → 0.78 KB flash, 0.15 KB SRAM (well within 256 KB flash / 64 KB SRAM budget)
- Inference time: <1.2 ms (negligible power impact)

4.3 Cloud Model (AWS)

- Random Forest classifier, tuned via GridSearchCV
 - Class weighting used to handle imbalanced hive condition classes
 - Not power/flash constrained → does detailed 6-way classification:
- Queenless / Cold Stress / Varroa Mites / Healthy Baseline / Imminent Swarming / Starvation Risk
 - Performance: 100% accuracy, perfect macro F1, weighted F1, AUROC on held-out test set
 - Any non-"Healthy Baseline" result triggers SMS alert to farmer

4.4 Why Two Models Instead of One

- Mirrors the edge/cloud hardware split
- TinyML: lower accuracy, but near-zero power, fully offline — always gives some read on hive state
- Cloud: higher accuracy, richer 6-class diagnosis — drives the actual alert
- Same design philosophy as hardware: keep always-on part lightweight, push heavy compute to where power/compute is available

5. Putting It Together (Summary Flow)

- Ansys simulations → validated RF, power, thermal behavior before build
- Simulation results → shaped physical PCB layout
- PCB → runs TinyML for instant offline classification

- Data also sent to AWS → cloud model does deeper 6-class diagnosis
- **Pipeline:** Sense → Simulate → Build → Infer → Alert

Sources: github.com/atharveeee-netizen/beevil-knievel (README.md, FINAL_REPORT.md, DATA_PROVENANCE.md, hardware/TinyML/Cloud Model folders), Ansys simulation slide deck